COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Advanced parallel and GPU algorithms

Standalone teaching module; use the workbook and separate answer key

MODULE 42 / FOUR LESSONS

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Module 42 — Advanced parallel and GPU algorithms 3

Lesson 42.1 — Parallel reductions, scans and work-span analysis 4

Lesson 42.2 — Tiling, data reuse and asynchronous pipelines 6

Lesson 42.3 — Irregular workloads, wavefront scheduling and load balance 8

Lesson 42.4 — Portable performance and integration-quality evaluation 10

Module 42 project: reusable GPU primitive in an engine 12

Primary reading directory 13

Module 42 — Advanced parallel and GPU algorithms

Prerequisites: Modules 18, 37-38 and 41.

Original project brief: Implement a reusable GPU reduction, scan or compaction primitive and integrate it into an engine workload. Compare a simple reference, a competent library baseline when available and your implementation at equal correctness requirements.

Core reference IDs: R37, R43, R48. Full entries are in the reading directory.

- 42.1 — Parallel reductions, scans and work-span analysis
- 42.2 — Tiling, data reuse and asynchronous pipelines
- 42.3 — Irregular workloads, wavefront scheduling and load balance
- 42.4 — Portable performance and integration-quality evaluation

Lesson 42.1 — Parallel reductions, scans and work-span analysis

Learning outcome: Prove the structure of a parallel primitive before tuning it.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A reduction combines many values with an operator; a scan produces prefix combinations. Associativity supports regrouping mathematically, but floating-point addition is not exactly associative. Decide whether the contract requires bitwise reproducibility, bounded error or an exact integer result. A tree reduction can reduce dependency depth from linear to logarithmic while retaining linear total work. Work is the total number of operations; span is the longest dependency chain. More processors cannot beat the span without changing the algorithm. A naive parallel scan can have extra logarithmic work, while work-efficient constructions achieve linear work with logarithmic span under their model. Handle partial groups, empty inputs and irregular sizes explicitly. Cross-group coordination must follow the API rather than assuming a global barrier inside an ordinary dispatch.

Work, span and the algebraic contract

Represent the computation as a directed acyclic graph. Work W counts all operations; span S is the longest dependency path. On P ideal processors the runtime cannot be below max(W/P,S). This is a lower bound in a declared model, not a hardware prediction: memory bandwidth, launch overhead, occupancy and barriers add constraints.

A reduction needs an identity and an associative operator for arbitrary regrouping. Integer addition modulo 2^32 is associative; real addition is associative; finite-precision floating-point addition is not exactly associative. Choose the numerical contract before changing order. Determinism on one fixed tree is weaker than agreement with every serial sum or every device. The Python scan uses arbitrary-precision integers, whereas the native and CUDA examples use uint32 wraparound. Their domains agree only when you account for that difference.

Derive exclusive scan through two tree passes

For inputs (3,1,7,0,4), an exclusive sum scan is (0,3,4,11,11). The value at position i is the reduction over positions strictly before i. In an upsweep, each tree node stores the sum of its children. Set the root to the identity, then walk downward: the left child receives the parent's prefix; the right child receives the parent's prefix plus the saved left-subtree sum. This preserves the left-to-right order even for associative operators that are not commutative. [S03]

Padding a non-power-of-two input with identities simplifies a teaching implementation. If the padded size is m, the two passes use 2(m-1) combines and logarithmic depth. Since n<=m<2n for n>0, work remains O(n). Do not apply log2(0); the empty scan returns an empty sequence. The exercise should establish those cases before adding dispatch-level hierarchy.

From scan to engine compaction

A stable compaction converts visibility flags to destination offsets using exclusive scan. Write each kept element once at its offset. The output count is the last offset plus the last flag for a nonempty input. Stability matters when later processing assumes original draw order or deterministic entity ordering.

A GPU hierarchy scans chunks, scans their totals in another dispatch and adds each chunk's offset. A normal thread-block barrier does not synchronize arbitrary blocks. Do not implement a global spin barrier that assumes all blocks can be resident. The optional CUDA reference instead performs multi-dispatch reduction, with out-of-range lanes supplying zero and every lane reaching each block barrier. It is supplied for hardware validation and was not compiled in this environment. [S04]

Worked engineering example

For n=1024, a complete binary reduction performs 1023 additions with 10 combine levels. An exclusive scan built from upsweep and downsweep on 1024 leaves uses 2046 additions in this simple model. With n=1025 padded to m=2048, it uses 4094 additions, still linear in n but almost twice the previous work for one extra input. That discontinuity is a reason to benchmark boundary sizes rather than only powers of two. For flags (1,0,1,1), offsets are (0,1,1,2); kept entries go to output slots 0,1,2.

Original worked check

A balanced reduction over 1,024 elements has 10 combination levels under a binary-tree model and 1,023 total pairwise combinations. Its implementation still pays memory and synchronization costs.

Practical assignment

Implement reduction and either scan or compaction. Pass: derive work/span bounds, test sizes 0, 1 and non-powers of two, and report the numerical/reproducibility contract.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_1.py`.

Reference starting point: code/gradlab/parallel.py; code/native/main.cpp; code/gpu/reduction.cu

Assigned study

Original assignment: R37: cooperative execution and synchronization; R48: parallel architectures; derive algorithm bounds using Module 14 methods.

Focused expansion: S03: GPU Gems 3, Chapter 39, work-efficient scan and large arrays; R37 / S04: CUDA SIMT programming and synchronization. Treat the historical chapter as algorithmic study, not current device tuning guidance.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. How many levels are in a balanced binary reduction of 1,024 values?
2. How many pairwise combines does that reduction require?
3. Why can parallel floating-point sums differ from serial sums?

Lesson 42.2 — Tiling, data reuse and asynchronous pipelines

Learning outcome: Overlap movement and computation without creating races.

Planning: 28-42 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Tiling partitions a computation so values can be reused in a faster memory level. Matrix multiplication, image filters and neighborhood operations benefit when the tile fits available storage and exposes sufficient parallelism. Larger tiles can increase reuse but also raise register or shared-memory demand and reduce residency. Double buffering can overlap preparation of one tile with computation on another. The pipeline needs explicit ownership: no consumer reads an incomplete tile and no producer overwrites data still in use. Asynchronous copy features vary by device; maintain a correct portable baseline. Account for boundary tiles, alignment, synchronization and extra staging work. Compare arithmetic intensity at the memory level actually being optimized.

Count reuse at the memory boundary you intend to optimize

A square B-by-B matrix-multiplication tile performs about 2B^3 arithmetic operations while loading two B-by-B input tiles for one inner-product stage. With 4-byte elements, that idealized stage moves 8B^2 input bytes, giving B/4 operations per input byte. This count deliberately excludes output traffic, boundary waste, staging and repeated loads at other memory levels. Never compare that optimistic intensity with a complete measured kernel as though the boundaries were identical.

For B=32, the two input tiles need 8192 bytes; double buffering them needs 16384 bytes before padding or additional metadata. Larger B increases reuse in the model but consumes more shared storage and registers. A tile that reduces active resident work or spills registers can lose even when the operation-to-input-byte ratio improves. Use measured bottlenecks to explain a result rather than merely saying "shared memory is faster." [R43, S04]

A pipeline is an ownership protocol

Name buffer states: free, filling, ready and consuming. The producer may reuse a slot only after the previous consumer finishes. The consumer may begin only after the copy's completion event, not merely after issue. Double buffering alternates slots, but alternating indices alone does not establish either dependency.

The reference schedule has one copy engine, one compute engine and two slots. For tile i, copy begins after both the copy engine and that slot are free. Compute begins after the tile is ready and the compute engine is free. This creates an executable timing/ownership model, not actual asynchronous hardware execution. A device implementation must translate every model edge into the API's synchronization contract and handle the first and last stage explicitly.

Prove correctness before measuring overlap

Boundary tiles need predicates for each load and store. Every thread participating in a block barrier must follow a compatible control path. Zero-fill out-of-range contributions rather than letting a few lanes return before a collective barrier. Include rectangular matrices such as 5x3 multiplied by 3x7 so that every dimension has a partial tile.

For a performance comparison, separate allocation, host preparation, transfer, kernel execution and result consumption. Then measure the end-to-end path directly: overlapping stage times cannot

simply be summed. A faster steady-state pipeline can still lose for one tile because fill/drain overhead dominates. Keep a non-overlapped implementation for correctness and unsupported hardware, and log why capability dispatch selected each path. [S04]

Worked engineering example

Assume a copy takes 2 time units and computation takes 5 per tile, with independent engines and two available slots. Eight serial tiles take 8*(2+5)=56 units. The ideal pipeline takes 2+8*5=42 units: copy tile 0, then let later copies overlap computation. The model speedup is 56/42=1.333x. This is a prediction under stated assumptions, not a measured GPU result. If staging adds 2 units to each compute stage, the same model becomes 2+8*7=58 units and loses to the original serial schedule.

Original worked check

For C=A*B, reusing a loaded A or B tile across many multiply-adds can raise useful operations per byte. A tile that spills heavily can erase that advantage.

Practical assignment

Implement a tiled matrix multiply or image filter, then one overlap variant where supported. Pass: adversarial dimensions match the reference, synchronization is checked and the end-to-end comparison includes staging overhead.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_2.py`.

Reference starting point: code/gradlab/parallel.py: tiled_gemm, pipeline_trace

Assigned study

Original assignment: R37: shared memory, synchronization and asynchronous operations where supported; R43: operational intensity.

Focused expansion: R37 / S04: shared memory and asynchronous execution; R43: original Roofline model. Read the chosen device API for stage completion and buffer-reuse semantics before attempting an asynchronous implementation.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can larger tiles hurt performance?
2. What ownership rule is essential for double buffering?
3. Why retain a non-asynchronous baseline?

Lesson 42.3 — Irregular workloads, wavefront scheduling and load balance

Learning outcome: Design around variable work rather than assuming identical threads.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Ray traversal, graph processing and variable-length simulation tasks have irregular work. A fixed assignment can leave some workers idle while a few long tasks dominate the tail. Work queues, compaction and wavefront processing reorganize active tasks, trading scheduling and memory traffic for better coherence or load balance. Persistent-worker designs keep workers alive to fetch tasks, but need bounded queues, termination detection and a memory-ordering contract. Global progress cannot depend on every workgroup being resident when the hardware does not guarantee that. Sorting tasks by similarity can improve coherence but has a cost and can alter latency. Evaluate both balanced and skewed distributions, including rare expensive tasks.

Throughput can look healthy while the last job dominates

For indivisible jobs with costs c_i on P workers, completion time is at least max(sum(c_i)/P,max(c_i)). A queue can redistribute jobs, but cannot make an indivisible 100-unit job finish in 10 units. Start by identifying whether a long task can be split: a ray path can move through bounded stages; a blocking external operation may not be divisible by your scheduler.

Static assignment minimizes scheduling overhead and can preserve locality. Dynamic assignment fills idle workers but adds atomic traffic, metadata movement and termination complexity. Distribution matters: uniform costs, a heavy tail and a spatially clustered hotspot can favor different strategies even at the same mean. Report makespan, queue depth and late-job latency in addition to operations per second.

Wavefront processing changes the unit of work

A monolithic path-tracing kernel carries different rays through intersection, material evaluation and secondary-ray generation. A wavefront approach places active work in stage-specific queues. This can improve coherence at a stage while increasing intermediate state traffic and dispatch cost. Compaction from Lesson 42.1 removes inactive entries between stages. Sorting by material or region adds yet another tradeoff. [S06]

For each queue, state capacity, producer count, consumer count and payload lifetime. Reserving a slot and publishing an initialized payload are different events. A consumer must not read the payload merely because an index was incremented if publication has not established the required ordering. A bounded queue must return a safe overflow/backpressure outcome, not silently overwrite live entries.

Termination needs more than an empty snapshot

An empty work queue does not prove the computation is finished while a worker can still publish descendants. One design tracks queued plus in-flight work under an ordering protocol; completion requires both to be zero with no new source admitted. Another bounds work by explicit dispatch stages and terminates when the next active count is zero. The correct choice depends on whether jobs create jobs and what progress the API guarantees.

Persistent GPU workers are not a license to spin on global barriers. Resident groups can occupy resources while waiting for groups that cannot start. Use an API-supported cooperative execution contract, a staged launch or a proof that does not require simultaneous residency. The included scheduling function models cost allocation only; it does not validate a concurrent queue, GPU memory ordering or global progress. Those remain implementation obligations. [S04]

Worked engineering example

For costs (100,1,1,...,1) with 99 one-unit jobs and 8 workers, total work is 199. The average-work lower bound is 24.875, but the indivisible-job bound is 100, so no scheduler in this model can finish below 100. Round-robin static assignment gives the first worker 100 plus 12 short jobs, for 112. A queue that starts the long job immediately can finish the rest before it, approaching 100 without dispatch overhead. Moving the long job late in queue order or charging queue costs changes the result. This is a workload model, not a measured GPU speedup.

Original worked check

If 99 tasks take 1 unit and one takes 100 units, static one-task assignments can leave the final completion dominated by the long task. Splitting or scheduling work differently may help only when the task is divisible.

Practical assignment

Implement an irregular workload using static assignment and a queue or compaction alternative. Pass: prove or test termination, handle queue capacity safely and measure tail behavior as well as mean throughput.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_3.py`.

Reference starting point: code/gradlab/parallel.py: scheduling_model; code/gradlab/rendering.py: BVH

Assigned study

Original assignment: R37: execution, atomics and synchronization; R48: multithreaded scheduling; R23: wavefront rendering as an application.

Focused expansion: R23 / S06: PBRT, Mapping Path Tracing to the GPU; S04: atomics and synchronization. Relate queue publication, consumption and completion accounting to the memory model studied in Module 18.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can average task cost hide poor completion time?
2. Why avoid a global spin barrier requiring every group to be resident?
3. Is sorting tasks always beneficial?

Lesson 42.4 — Portable performance and integration-quality evaluation

Learning outcome: Separate algorithmic benefit from one device-specific tuning win.

Planning: 22-34 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A performance-portable implementation preserves correctness and obtains useful efficiency across its stated target set. This does not mean identical code or identical speed everywhere. Capability dispatch can select specialized paths while keeping a tested fallback. Tuning parameters should be bounded and recorded rather than silently selected from the final test set. Compare against a competent baseline and include small, large and irregular workloads. Report speed, memory, numerical error and integration cost. A kernel that is faster only after excluding transfer or conversion work should make that boundary explicit. Reproducible artifacts include build scripts, device capability reports, raw data and failure cases. Lack of access to multiple GPUs is a limitation to disclose, not a reason to invent portability evidence.

Portability has at least three separate questions

Functional portability asks whether the implementation is valid on the supported target set. Performance portability asks whether it retains useful efficiency across that set. Artifact portability asks whether another engineer can build and run it. One source file that compiles on two devices answers none of these questions completely.

Describe the target set explicitly: API, feature level, precision, storage limits and subgroup assumptions. Specialize only behind capability checks. A tile fitting on device A may be invalid on B, even before performance is considered. Avoid using a single vendor's execution width as a language-level invariant. Keep a baseline that satisfies the same numerical contract and report the reason for every fallback. [S04, R21]

The application upper bound comes before heroic kernel work

Suppose a primitive accounts for fraction f of total baseline time and is accelerated by s, with all other work unchanged. The ideal application speedup is 1/((1-f)+f/s). This derives directly from adding the unchanged and accelerated fractions; it excludes integration overhead. For f=0.2 and s=4, the maximum under those assumptions is 1/0.85, approximately 1.176x. Any added data conversion reduces that benefit.

Roofline provides a different bound: attainable arithmetic rate is no more than min(peak arithmetic rate, measured bandwidth times arithmetic intensity) under its model. Combine the two views rather than using either as a proof of achieved speed. A kernel can be bandwidth-limited while also being unimportant to the frame's critical path. [R43]

A fair comparison is a maintained artifact

Use a competent library baseline when available and disclose its version and configuration. Separate allocation, transfers, conversion, synchronization and computation; then time the end-to-end operation as used by the engine. Report small workloads where launch overhead dominates, large workloads where bandwidth matters, and irregular workloads that expose boundaries.

An untested device is a blank evidence cell, not a predicted pass. When only one GPU is available, evaluate multiple workloads on that device and narrow the claim. A CPU model can verify indexing or algebra, but cannot certify GPU performance portability. The package's native benchmark deliberately compares two CPU algorithms and preserves losses; it is a training artifact for measuring honestly, not a sales claim about a universal winner.

Worked engineering example

A frame takes 10 ms: 2 ms is the selected primitive and 8 ms is other work. A 4x primitive produces 8+0.5=8.5 ms, giving 1.176x application speedup. Now add 0.7 ms of packing and 0.4 ms of synchronization: the frame is 9.6 ms, only 1.042x faster. A report showing only the 2-to-0.5 ms kernel chart hides almost all of the engineering conclusion. If a different scene spends just 0.2 ms in the original primitive, the same fixed integration cost can turn the candidate into a regression.

Original worked check

A tile size tuned on one GPU can reduce occupancy or exceed shared-memory limits on another. Capability checks and independent tuning prevent that choice from becoming an invalid universal default.

Practical assignment

Integrate the selected primitive into the engine and evaluate at least two workload regimes. Pass: application impact is measured, unsupported features fall back safely and any untested device claims are explicitly excluded.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_4.py`.

Reference starting point: code/native/main.cpp --bench; templates/CAPABILITY_MATRIX.md

Assigned study

Original assignment: R37 and R21: capability contracts; R43: performance models; Module 41 experimental protocol.

Focused expansion: R43: Roofline model; S04: capability and launch constraints; R21: the selected graphics/compute API capability contract. Use only measured hardware limits for performance estimates.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Does portable performance require one fixed launch configuration?
2. What should be said when only one GPU was tested?
3. Why measure integration cost?

Module 42 project: reusable GPU primitive in an engine

Implement reduction plus scan or compaction, with a declared exact-integer or floating-error contract. Derive work/span, handle empty and partial groups, and use supported inter-dispatch synchronization. Integrate the primitive into an engine workload such as visible-instance compaction, particle filtering or task queue preparation.

Compare a simple independent reference, a competent available library primitive and your implementation. Include small, large and irregular sizes, boundary alignment, dispatch/setup, transfers and application consumption. Record actual target capabilities and fallbacks. A second GPU strengthens portability evidence; without one, state that only the tested device is verified. A CPU model cannot become the missing second device.

Required artifacts: API contract, device capability record, correctness/sanitizer logs where supported, before/after engine trace, raw paired timings, error results and portability matrix. Correctness and synchronization 40 points, integration 25, performance method 20, portability/limitations 15. Suggested pass: 80/100 and no race, numerical-contract violation or fabricated device result. Device evidence pending means the hardware part is not yet passed.

Primary reading directory

R01-R55 retain the original program IDs. S01-S17 are focused additions for this expansion. Citations such as [S05] refer to the entries below; equations and worked cases not attributed to another source are original teaching derivations. External resources retain their own rights and are linked rather than redistributed. Moving documentation is not a promise of compatibility with an untested installed version. Record exact versions for your own artifact.

R21 — Vulkan tutorial

Khronos Group

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R23 — Physically Based Rendering, fourth edition

Matt Pharr, Wenzel Jakob, Greg Humphreys

https://www.pbr-book.org/4ed/contents

Rendering mathematics and implementation; use chapter titles in lesson assignments.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R37 — CUDA Programming Guide

NVIDIA

https://docs.nvidia.com/cuda/cuda-programming-guide/

GPU execution and memory; CUDA is an optional vendor-specific lab route.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R43 — Roofline performance model, technical report

Samuel Williams, Andrew Waterman and David Patterson

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

Original 2008 performance-model reference; use current measured hardware limits for experiments.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R48 — 6.823 Computer System Architecture

MIT OpenCourseWare

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

Graduate architecture foundations; historical course concepts supplemented with current specifications.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

S03 — Parallel Prefix Sum (Scan) with CUDA, GPU Gems 3, Chapter 39

Mark Harris, Shubhabrata Sengupta and John D. Owens

https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda

Work-efficient scan foundations; historical algorithms, not current tuning defaults.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S04 — CUDA Programming Guide

NVIDIA

https://docs.nvidia.com/cuda/cuda-programming-guide/

Execution and memory contracts; moving documentation consulted, no matching toolkit/device validation claimed.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S06 — PBRT 4e: Mapping Path Tracing to the GPU

Matt Pharr, Wenzel Jakob and Greg Humphreys

https://www.pbr-book.org/4ed/Wavefront_Rendering_on_GPUs/Mapping_Path_Tracing_to_the_GPU

Wavefront organization and queue-oriented rendering.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.